



华 映 资 本

LFM2-VL

论文图解



核心要点

- 为什么值得关注：边缘级 VLM 的部署挑战与 LFM2-VL 的定位。
- 核心创新：原生分辨率、动态切片、轻量连接器与可调 token 预算。
- 逐图解读：图 1 Demo、图 2 架构、图 3 速度、图 4 显存。
- 评测与选型：benchmark 结果如何看，450M 与 1.6B 如何取舍。
- 结论：这条路线对端侧多模态系统意味着什么。

真正难的不是做出 VLM，而是在端侧把它跑得又快又稳。

边缘级 VLM 的三重约束

LFM2-VL 试图同时解决速度、内存和灵活推理三个问题。

低延迟

终端问答与拍照理解需要快速响应，不能把多模态推理全部留给云端。

低资源占用

显存、内存和能耗都受限，传统大体量 VLM 很难直接落在手机和边缘盒子。

可调推理成本

不同图像分辨率和任务复杂度，需要可以在线调节的 token 预算。

核心要点

- 低延迟：拍照问答、现场巡检、AR 交互都要求即时响应。
- 低资源：手机、笔记本和边缘盒子对内存与功耗都很敏感。
- 灵活推理：不同图像大小和任务复杂度，不该被固定输入流程绑死。
- LFM2-VL 的价值就在于：把“能跑起来”也当成模型设计目标。

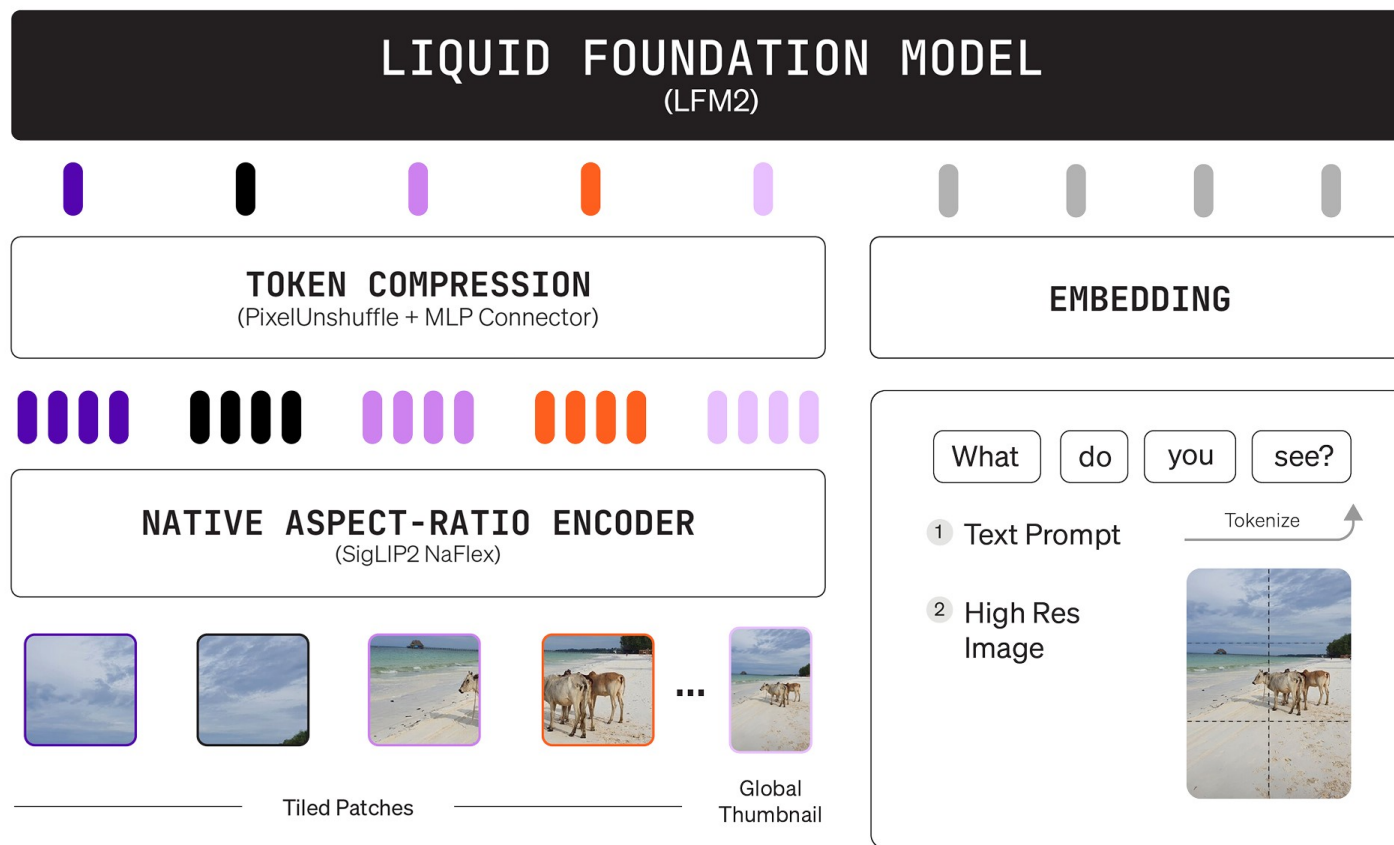
核心创新：把端侧 VLM 做成可调节的 token 经济学

原生分辨率、动态切片和轻量连接器共同服务部署效率。

核心要点

- ◆ 基于 LFM2 语言主干扩展到视觉语言，而不是重新堆一个更重的骨干。
- ◆ 图像原生处理到 512x512，避免为了统一输入而强行上采样。
- ◆ 大图按 512x512 patch 切片，并用缩略图保留整体场景语义。
- PixelUnshuffle + 2-layer MLP 先压缩视觉 token，再送入语言模型。

“A sandy white beach with three cows facing the ocean”



先补背景： LFM2 给 LFM2-VL 提供了什么底座

没看过 LFM2 也只要记住，它本身就是面向效率优化的语言模型。

核心要点

- LFM2 主干强调短卷积与少量 GQA 的混合设计，目标是兼顾表达能力与推理速度。
- ♦ 这意味着 VLM 部分继承的是一个本来就面向端侧和低延迟优化过的语言 backbone。
- LFM2-VL-450M 使用约 86M 的 SigLIP2 NaFlex base 编码器，优先考虑更紧的资源预算。
- LFM2-VL-1.6B 使用约 400M 的 shape-optimized 编码器，换来更强的细粒度视觉能力。
- ♦ 两个版本共享同一套方法论：先把成本控住，再尽量把视觉信息保留下来。

逐图导览：这篇工作主要靠哪几张图说服我们

四张图分别回答：能做什么、怎么做、快多少、省多少。

核心要点

- ◆ 图 1：用一个直观 demo 展示 450M 模型的图像理解与语言描述能力。
- ◆ 图 2：解释视觉编码、token 压缩和 LFM2 主干如何拼成完整的 VLM。
- ◆ 表 1：把精度放回参数量和部署成本语境里，观察它是否“对得起体量”。
- ◆ 图 3：速度对比证明其 GPU 推理时间能明显压低到更适合交互应用的区间。
- ◆ 图 4：显存对比说明它不是绝对最小，但在效率 - 能力间找到了更平衡的位置。

图 1 解读：一个小 demo 实际上在展示什么

它不是 benchmark，但很直观地展示了 450M 版本的基本能力边界。

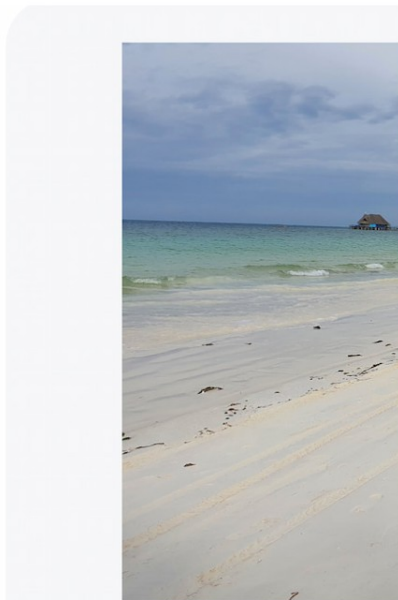


Figure 1 demo 的直观信息

模型不仅回答了“这是海滩”，还识别出前景中的牛，并把场景组织成一段完整自然语言描述。

这说明 450M 版本已经具备基础视觉 grounding、场景概括和文字生成的一体化能力。

核心要点

- ♦ 模型先识别出海滩场景，再抓住“牛在海边”这种相对少见的关键细节。
- ♦ 输出不是单标签分类，而是完整自然语言描述，说明视觉 grounding 与生成是连通的。
- ♦ 论文故意选 450M 版本做展示，强调的正是“小模型也能完成有用视觉理解”。
- ♦ 当然，这类 demo 更像能力切片，真正可靠性还要回到后面的 benchmark 与效率图来看。

图 2 解读：LFM2-VL 的整体架构是怎样工作的

核心不是把图像硬塞进 LLM，而是先把视觉信息结构化压缩。

核心要点

- ◆ 底层语言推理由 LFM2 backbone 承担，视觉部分只负责把图像变成可对齐的 token。
- ◆ SigLIP2 NaFlex 编码器既支持原生分辨率输入，也支持非标准长宽比，不必先统一缩放。
- ◆ 中间的 connector 通过 PixelUnshuffle + MLP 降低 token 数量，再和文本 token 共同进入 LFM2。
- ◆ 这让模型的“重活”更多集中在有效信息而非冗余像素上，是效率优势的关键来源。

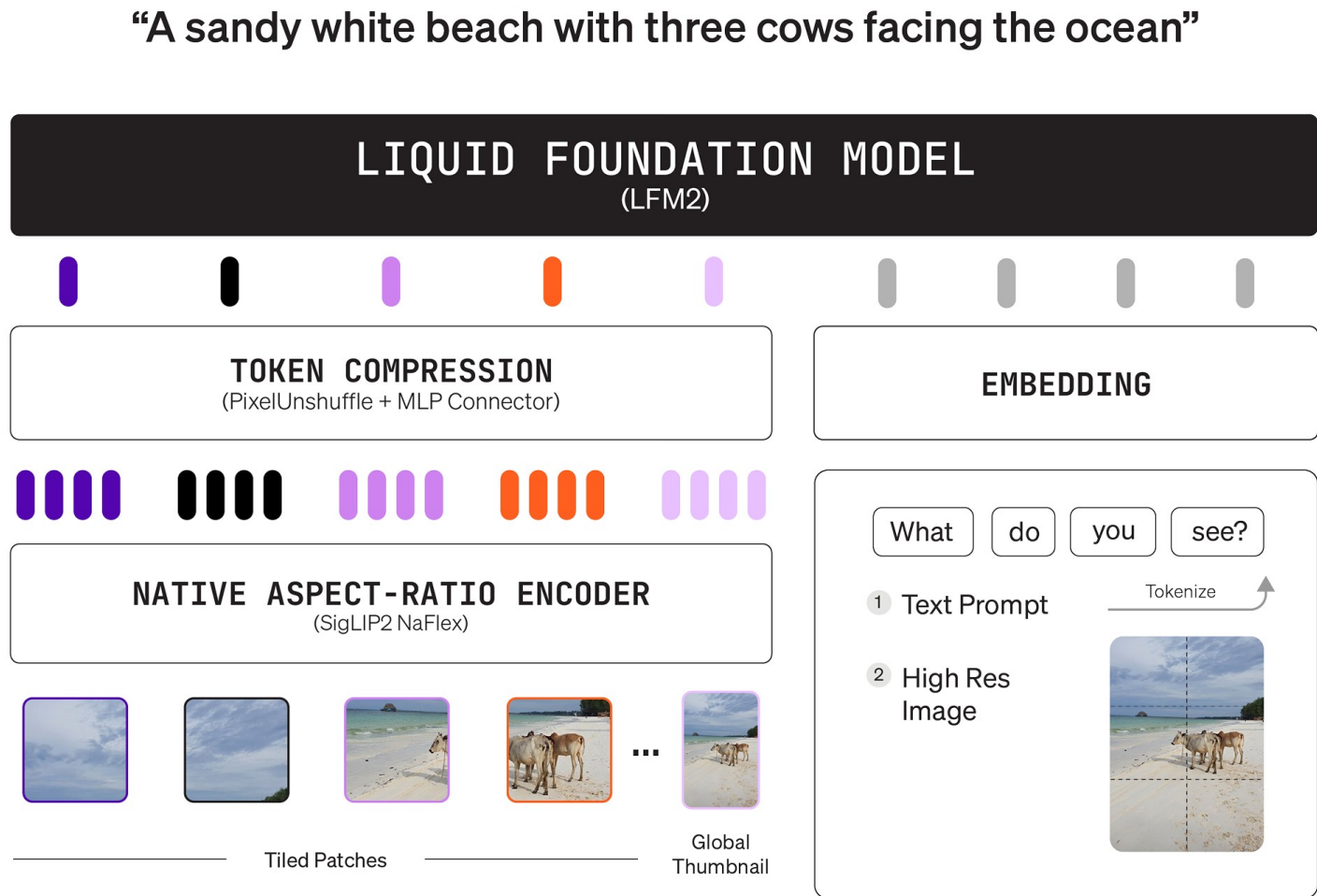


图 2 继续：高分辨率为什么还能跑得动

关键在动态切片、全局缩略图，以及可在线调节的视觉 token 预算。



论文例子：256x384 -> 96 tokens, 384x680 -> 240 tokens, 1000x3000 -> 1020 tokens。

核心要点

- 小图直接按原生尺寸处理，避免先放大再编码带来的额外开销。
- 大图会被切成多个 512x512 patch；1.6B 版本还会额外看到一个全局 thumbnail。
- 论文给出的例子是：256x384 约 96 tokens，384x680 约 240 tokens，1000x3000 约 1020 tokens。
- 用户在推理时可以直接调最大图像 token 数和 patch 数量，不必重新训练模型。

评测结果：不是绝对 SOTA，但非常对得起体量

要把精度放回参数规模与部署成本语境里，才能看懂这篇论文的卖点。

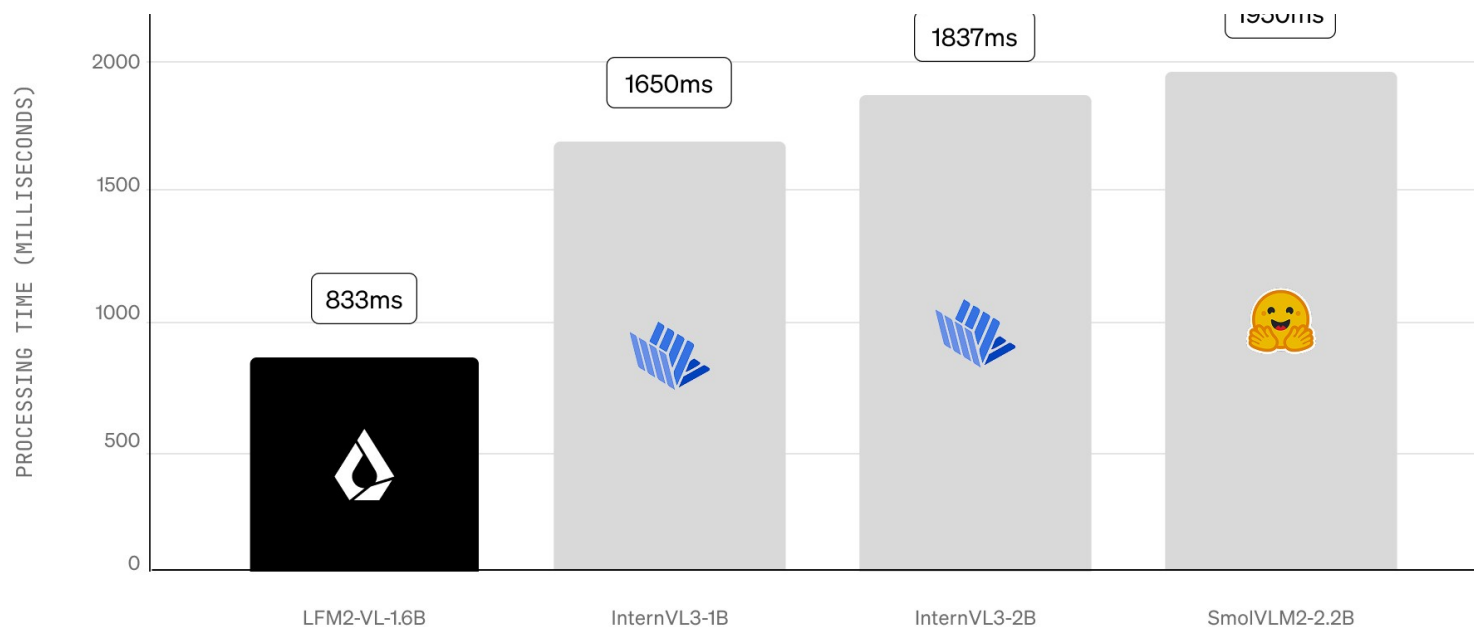
任务	LFM2-VL-1.6B	LFM2-VL-450M	对比 2B 级模型
RealWorldQA	65.23	52.29	65.10
InfoVQA (Val)	58.68	46.51	66.10
OCRBench	742	655	831
MathVista	51.10	44.70	57.60
SEEDBench_IMG	71.97	63.50	75.00

核心要点

- 1.6B 版本在 RealWorldQA 上到 65.23，已经接近 2B 级 InternVL3-2B 的 65.10。
- OCRBench、MathVista、SEEDBench_IMG 等结果说明它在 OCR、高分辨率和推理类任务上并不掉队。
- 450M 版本不是拿来冲榜的，而是给手机、穿戴和更苛刻设备预算准备的可用选项。
- ◆ 所以论文真正强调的是“足够好的精度 + 明显更好的效率”，而不是一味追求最高分。

图 3 解读：速度优势到底有多实在

在相同任务设定下，它把交互延迟直接压到了更适合产品落地的区间。



核心要点

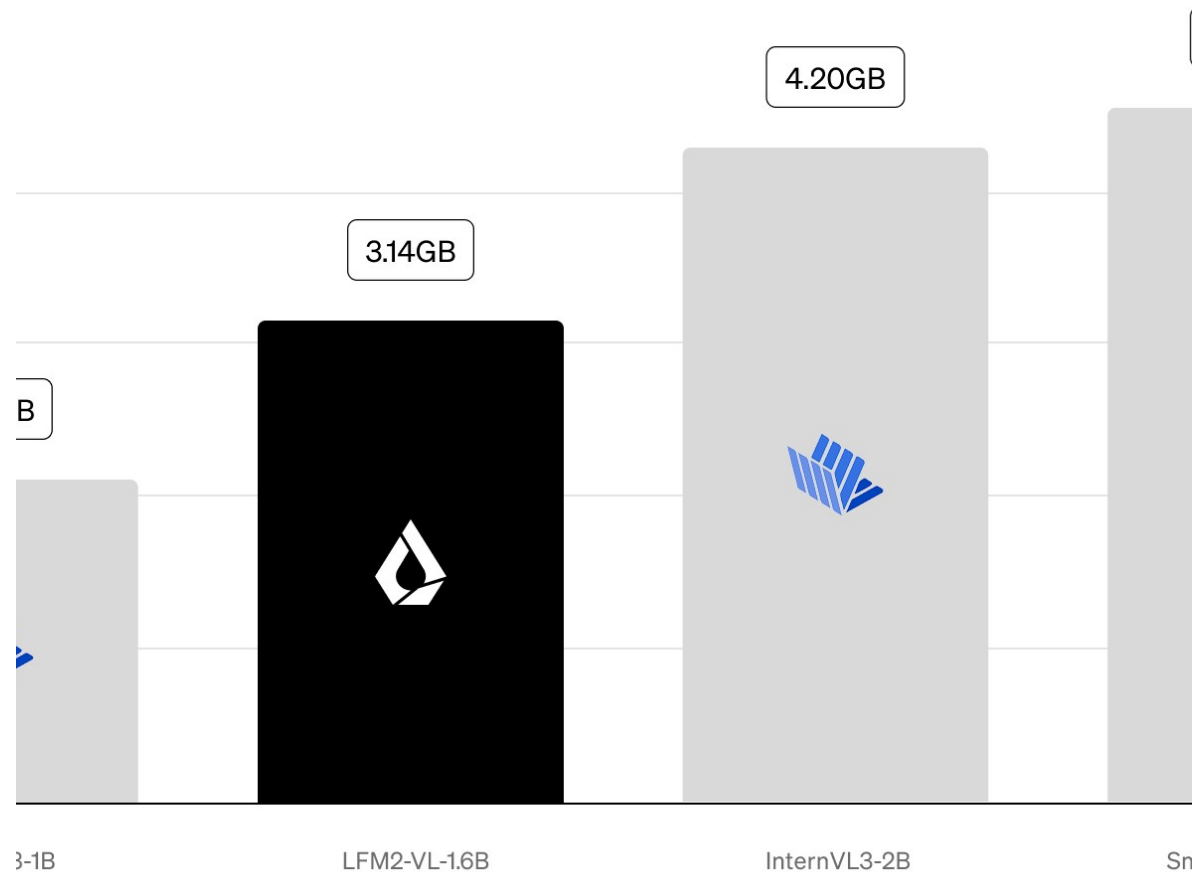
- ♦ 论文设定的 workload 是：1 张 1024x1024 图像 + 简短提示词 + 生成 100 个 token。
- ♦ LFM2-VL-1.6B 的处理时间约 833ms，而几款对比模型分别在 1650ms、1837ms 和 1950ms 左右。
- ♦ 这意味着它相对最快竞品也接近 2 倍速度提升，优势不是小修小补，而是交互层面的体感差异。
- ♦ 这张图最能说明：token 压缩和高效 backbone 的设计，最终都会兑现成真实等待时间的减少。

图 4 解读：显存并非最小，但性价比很高

要理解这张图，重点不是“谁最低”，而是“谁在更强能力下仍没把成本拉爆”。

核心要点

- LFM2-VL-1.6B 约 3.14GB，低于 2B 级对手的 4.20GB 和 4.62GB，但高于更小的 1B 模型 2.06GB。
- ◆ 也就是说，它不是绝对最省内存的模型，但在参数规模、速度和视觉能力之间更平衡。
- ◆ 对笔记本、本地工作站和单 GPU 场景，这种“稍高一点显存换来更强能力”的交易通常是划算的。
- ◆ 反过来，如果预算极紧，450M 版本才是更符合边缘设备约束的选择。



先按设备预算分层，再看任务是否依赖细粒度视觉理解。

450M 搭载更轻的视觉编码器，更适合手机、穿戴、边缘盒子这类强调实时性与低资源占用的场景；1.6B 则更适合笔记本、单 GPU 和需要更强 OCR / 高分辨率理解的任务。

LFM2-VL-450M

更轻量

手机 / 穿戴 / 边缘盒子

LFM2-VL-1.6B

更强细节

笔记本 / 单 GPU / 本地工作站

两者共享同一套 token 预算调节机制，所以推荐的选型顺序是：先看内存与延迟，再判断任务是否依赖细节，最后再调图像 token 上限和 patch 数量。

综合讨论：这篇工作真正贡献了什么

它把端侧 VLM 的优化目标说清楚了：不是更大，而是更可部署。

核心要点

- ◆ 这篇论文最有价值的地方，不是某个单点分数，而是把“速度、显存、精度”一起纳入设计目标。
- LFM2-VL 说明高分辨率视觉理解不一定等于固定高成本，token 预算本身可以成为可调控制杆。
- ◆ 对工程侧来说，这比单纯追求 benchmark 榜首更有意义，因为它直接影响产品是否能落地。
- ◆ 真正的启发是：未来端侧多模态系统的竞争，很可能是体系化效率设计的竞争。

LFM2-VL 代表了一条更务实的边缘多模态路线。

核心要点

- ◆ 图 1 告诉我们：即便是 450M 级别，小模型也能完成有用的视觉 grounding 与描述。
- ◆ 图 2 给出的配方是全篇核心：NaFlex 编码器、动态切片、thumbnail 和 token 压缩共同协作。
- ◆ 图 3 和图 4 进一步证明：效率收益是真实存在的，而且能直接转化成更短等待时间与更低部署门槛。
- ◆ 如果目标是做边缘侧多模态系统，LFM2-VL 的最大启示是“可控成本”往往比 headline SOTA 更重要。